Дослідження all_of, варіант 1

Дослідження all_of виконано у Windows x64 з MSVC 19.51, C++20 та 16 логічними потоками процесора. Послідовності з 1 000, 100 000 і 1 000 000 чисел згенеровано через mt19937 із зерном 42 та збережено у файлах, спільних для всіх режимів. Перевірялися додатність і простота числа у випадках з усіма істинними значеннями та хибним елементом на початку або наприкінці.

Обидва режими були Release: /Od та /O2 /Ob3 /GL /LTCG. Час визначався через steady_clock як медіана п'яти повторів після прогріву, без читання й виведення. Власний алгоритм запускав std::all_of у K окремих потоках для приблизно рівних частин даних, потім зводив результати через std::all_of. Створення потоків, очікування та зведення входили до часу. Перевірялися K = 1, 2, 3, 4, 8, 16, 32 і 64.

Коли всі елементи задовольняли предикат, збільшення обсягу даних збільшувало час послідовного проходу. З оптимізацією перевірка додатності без політики для трьох довжин тривала 0,0017, 0,129 і 1,310 мс відповідно. Перевірка простоти тривала 0,136, 12,228 і 141,250 мс. Перебір дільників потребує більше обчислень на кожен елемент, тому паралельна обробка для нього має більше користі.

Для мільйона елементів з оптимізацією seq та unseq дали 1,573 і 1,433 мс для додатності та 166,965 і 156,645 мс для простоти. Політики par та par_unseq дали 0,333 і 0,277 мс для додатності та 14,792 і 14,569 мс для простоти. Отже, par_unseq прискорила перевірку простоти у 9,695 раза проти алгоритму без політики. Для малої послідовності з 1 000 елементів перевірка додатності без політики була швидшою за найкращий власний запуск: 0,144 мс при K = 1.

Рівень оптимізації також вплинув на результати. Для мільйона елементів час перевірки додатності без політики змінився з 2,284 мс без оптимізації до 1,310 мс з оптимізацією, а час перевірки простоти змінився з 153,530 до 141,250 мс. Власний алгоритм без оптимізації досяг найкращого часу для цих випадків при K = 8 і K = 16, з часом 0,718 і 16,496 мс відповідно.

З оптимізацією найкращий перевірений K для мільйона елементів становив 4 для додатності та 16 для простоти. Це 0,250 і 1,000 від кількості логічних потоків процесора. Час дорівнював 0,513 і 16,119 мс, що відповідає прискоренню у 2,554 і 8,763 раза проти алгоритму без політики. Оптимальне K залежить від обсягу даних, предиката та режиму оптимізації. Тут воно обране лише серед перевірених значень, тому глобальний оптимум не гарантується.

Розташування хибного елемента змінювало обсяг роботи. Якщо він був першим, послідовний алгоритм завершувався майже одразу: для мільйона елементів і перевірки простоти з оптимізацією час становив 0,0001 мс. Власний алгоритм у найкращому запуску витратив 0,067 мс, оскільки інші частини продовжували роботу незалежно від першої. Коли хибний елемент був останнім, послідовний алгоритм переглядав усю послідовність, а найкращий власний запуск тривав 15,542 мс. Дуже короткі вимірювання біля межі таймера не дають надійної оцінки точного прискорення.

Збільшення K спочатку дозволяє розподілити обчислення, але далі зростають витрати на створення потоків, очікування та об'єднання результатів. Для мільйона елементів з оптимізацією час перевірки додатності при K = 16, 32 і 64 становив 0,715, 1,069 і 1,883 мс. Для простоти відповідні значення становили 16,119, 17,051 і 16,354 мс. Накладні витрати створення й об'єднання зростають приблизно лінійно з K, проте загальний час також залежить від планувальника та завантаження процесора. Єдиного точного закону зростання ці вимірювання не встановлюють.

Паралельна обробка найбільш корисна для великих послідовностей і дорогих предикатів. Для коротких перевірок або раннього хибного елемента послідовний запуск часто вигідніший. Повні результати збережено у results/lab2_v1_O0.txt та results/lab2_v1_O2.txt.